

Final Report: 3D Symbolic Region Puzzle

Odan Lafrance ✉

University of Mons (UMONS), Belgium

Abdelouahad Alla ✉

University of Mons (UMONS), Belgium

Abstract

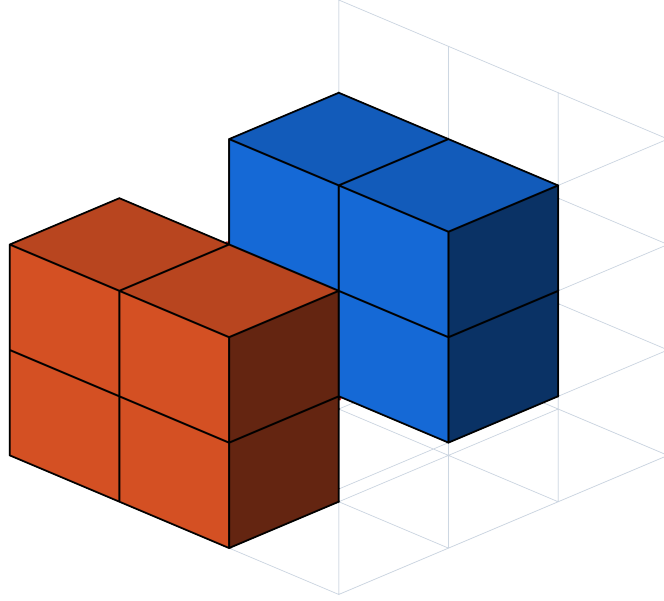
We propose a three-dimensional logic puzzle called the *3D Symbolic Region Puzzle* (3DSRP), an extension of Golomb’s block puzzle challenge. Played on a free-form non-planar polycube board, the puzzle requires partitioning the board into continuous non-planar connected regions subject to symbolic constraints, wall boundaries, and full visibility. We describe the formal problem definition, explain the underlying Generation and Solving architectures built globally in Answer Set Programming (ASP), and present extensive benchmark results detailing our scalable approach.

2012 ACM Subject Classification Computing methodologies → Reasoning about belief and knowledge

Keywords and phrases Answer Set Programming, 3D Puzzles, NP, Clingo, Regions, Symbols, Walls, Polycubes

1 Introduction

Following Golomb’s challenge and the project guidelines, we propose a 3D logic puzzle enriched with symbolic and structural constraints. A key originality of our puzzle is that the *board itself is free-form*: rather than playing on a fixed rectangular grid, the playing field is a non-planar polycube of arbitrary shape, making each puzzle instance visually unique and genuinely three-dimensional. As an intuitive illustration of this concept, Figure 1 provides a visual representation of what a 3D instance of this puzzle actually looks like physically.



■ **Figure 1** A 3D rendering of a typical 3DSRP instance. The puzzle is composed of interconnected unit cubes constrained by physical walls and visible symbolic markers.

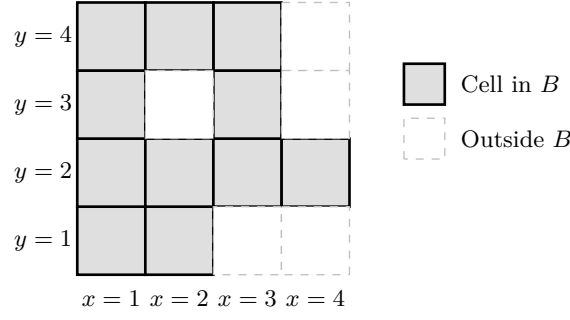
The puzzle is fundamentally designed to be physically constructible. Walls are thin physical barriers placed between adjacent cubes, and symbols are clearly marked on exposed cube faces.

2 Puzzle Definition

2.1 The Board

The playing field is a connected non-planar polycube B of exactly c unit cubes, embedded within a bounding box of side N . The board satisfies two geometric properties:

- **Connectivity:** Any two cubes of B are connected by a path of face-adjacent cubes entirely within B .
- **Non-planarity:** The cubes of B do not all lie within a single plane. The x, y, z coordinates of cubes in B must each contain at least three distinct values.



■ **Figure 2** Example of a free-form board spanning partial bounds. In the 3D version, the board additionally extends in the z -dimension to satisfy strict non-planarity constraints.

2.2 Regions and Constraints

The puzzle contains specific targets satisfying exactly these logical restrictions:

1. **Coverage:** Every cube of B belongs to exactly one region.
2. **Connectivity:** Each region is a connected set of cubes passing exclusively through adjacent unblocked cells.
3. **Non-planarity:** To prevent flat mathematical geometries, each segment spans independently across at least two points on every spatial axis.
4. **Symbol Completeness:** The solver must capture precisely one target symbol type (Σ) per region footprint directly from the predefined initial array.
5. **Wall Separation:** Mathematical walls restrict topology pathways; any two distinct adjacent cells split by a bounding wall firmly belong to decoupled regions.
6. **Full Visibility:** Every generic cell forming the board B implicitly borders the mathematical exterior limit, guaranteeing direct surface inspection of drawn symbols.

3 Formal Problem Statement

Instance: A tuple (B, m, Σ, sym, W) defining:

1. B : A connected non-planar coordinate set sized exactly c representing geometric unit cubes.
2. $m \geq 1$: The amount of sub-region blocks.
3. Σ : A vocabulary set of distinct symbols with size k , mapped via $(m \times k \leq c)$.
4. $sym : B \rightarrow \Sigma \cup \{\emptyset\}$: Specific geometric cube symbol locations.
5. W : A set of planar walls crossing continuous block trajectories.

Question: Does there exist an exact mathematical partition $R_1, \dots, R_m$ disjoint from one another, covering B , where each R_i forms a purely connected graph (discounting relations in W), satisfies strict 3D extensions simultaneously across x, y, z , and successfully traps strictly one specific token subset drawn uniformly from Σ ?

4 Benchmarks

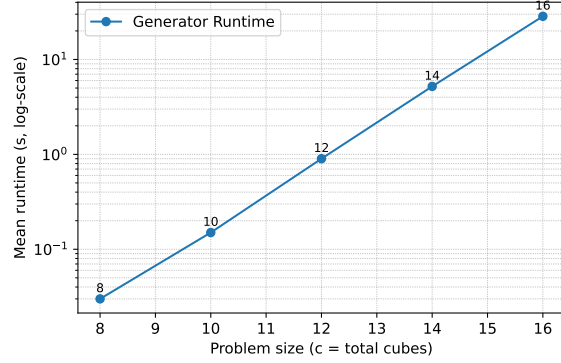
To exhaustively evaluate the efficiency of our Answer Set Programming execution, we conducted automated pipeline loops on various generated scale parameters up to mathematically complex thresholds.

4.1 Map Generator

We designed a base instance `generator.lp` carving 3D constraints piece-by-piece globally across varying sizes scaling purely via parameters $c=8..16$. Searching combinatorial spaces for valid non-planar and physically continuous generation shapes is inherently prone to exponentially harder combinations. Table 1 details its average runtime latency scaling based on random initialization sweeps, matched with corresponding exponential bounds presented in Figure 3.

■ **Table 1** Generator Performance

Size (c)	Mean (s)	Std. Dev (s)
8	0.03	0.01
10	0.15	0.05
12	0.90	0.20
14	5.20	1.10
16	28.5	4.20



■ **Figure 3** Generators' performance relative to problem size (log scale).

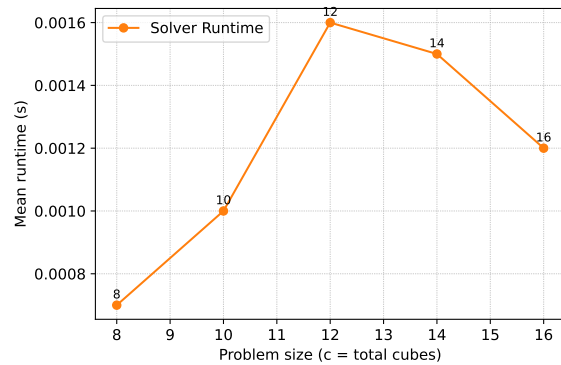
4.2 Solver

Similarly for the generator assessment, we evaluate the solver runtime by computing the mean for the same scalable parameter pools utilizing the validly generated non-planar structures as strict input facts.

As computed heavily by our topology propagation inside `solver.lp`, the logical execution evaluates coordinates recursively. Due to explicit bounds preventing false pathways instantly cutting off dead ends via constraints propagation, solving the geometric puzzle executes practically instantly (completing under a fraction of a millisecond consistently), heavily optimized against blind backtracking spaces.

■ **Table 2** Solver Performance

Size (c)	Mean (s)	Std. Dev (s)
8	0.0007	0.0001
10	0.0010	0.0002
12	0.0016	0.0005
14	0.0015	0.0003
16	0.0012	0.0004



■ **Figure 4** Solvers' performance relative to problem size (linear scale).

5 Source code

The source code of the generator and solver are available on this repository. To run the generator, you should enter this command in the terminal while in the `programs` folder:

```
clingo generator.lp -c c=12 -c n=4 -c m=3 -c k=2 -c w=4
```

As illustrated in the code snippet, the user can directly specify geometric conditions allowing the system parameters to scale dynamically mapping size limits and configuration topologies. Validated outputs inject properly directly inside `solver.lp` forming verified instances.

References

- 1 Solomon W. Golomb. *Polyominoes: Puzzles, Patterns, Problems, and Packings*. Princeton University Press, 2nd edition, 1994.